

DBMS Assignment 8

Instructions:

20-03-2026

Submit one PDF and one Jupyter Notebook (.ipynb) file.

- The pdf file should contain the question, query/code, outputs, and explanation.
- The name should be <roll no>_Assignment8.pdf, <roll_no>_Assignment8.ipynb
- Complete the exercise before 4.45 PM and submit the files in Moodle.
- Marks Distribution for both Postgres and Python (query/code + output+explanation)

Trigger variable:

TG_OP is a built-in trigger variable in PostgreSQL that stores the type of operation (INSERT, UPDATE, DELETE) that fired the trigger.

SYNTAX:

```
IF TG_OP = 'INSERT' THEN
    -- code for insert
ELSIF TG_OP = 'UPDATE' THEN
    -- code for update
ELSIF TG_OP = 'DELETE' THEN
    -- code for delete
END IF;
```

Question 1: (10 Marks)

Create a table: **employee(emp_id, emp_name, department, salary)**

Create another table: **employee_audit(audit_id, emp_id, operation_type, change_time)**

a) Write a **trigger function** that logs every **INSERT and UPDATE** operation performed on the employee table. (2)

b) The trigger should record:

- employee ID
- operation type (INSERT / UPDATE)
- timestamp of the operation. (1)

c) Insert the following records: (1)

(201,'John','IT',50000)
(202,'Mary','HR',45000)
(203,'David','Finance',52000)

d) Update the salary of employee **Mary** to **48000**.(1)

e) Modify the trigger such that when an employee's salary is updated, the audit table also stores:

- previous salary
- new salary

Update the salary of employee **John** and display the audit table.(2)

f) Extend the trigger such that whenever an employee record is **deleted**, the operation is logged in the employee_audit table with operation_type = 'DELETE'.

Test the trigger by deleting employee **David**.(2)

g) Identify the **triggering event** in each of the following cases:(1)

- Logging new employees
- Updating employee salary
- Deleting an employee

State whether each corresponds to:

- INSERT event
- UPDATE event
- DELETE event

Display the contents of the **employee_audit** table.

Question 2 : Using the dvdrental database.(5 Marks)

Create a trigger that **prevents inserting a payment** if:

- the **staff_id** does not exist in the staff table
- OR the **payment amount** is negative.

- a) Write the trigger function.(2)
- b) Create the trigger on the **payment table**.(1)
- c) Test the trigger using the following cases:(1)

Example 1 – Valid payment insertion.

Example 2 – Invalid **staff_id**.

Example 3 – Negative payment amount.

- d) Explain briefly why **BEFORE triggers are more suitable for validation tasks**.(1)

Question 3: Execute the following queries using **two approaches**:(7 Marks)

- 1. Direct SQL execution in PostgreSQL.
- 2. Fetch rows using **Python** and perform the processing in Python.
(Use **psycopg2** to connect Python to PostgreSQL.)

Use:

- **EXPLAIN ANALYZE** for PostgreSQL
- **time** module in Python

Compare the **execution time** for both approaches.

- a) Retrieve the **details of the 4th highest paying customer** based on total payments.(1+1)
- b) Display a list of **films that have never been rented**.(1+1)
- c) Find the **top 5 customers who rented the most films**.(1+1)

Show the execution time for each method and compare the results.(1)

Question 4 :Insert data in bulk from the insert_countries.csv file into the country table **using Python**. Ensure that after every 3 insertions, the data is persisted in the database. Display the table contents after each persisted insert.(5)

Question 5: (8 Marks)

- a) Using Python (psycopg2 / SQLAlchemy class), perform a transaction on the payment table:(3)
 - Insert a new payment for a customer
 - Update the same customer's payment
 - Calculate total payment (SUM) before and after the transaction
 - Commit if successful, otherwise rollback

- b) Write a SQL query to: (2)

- Calculate **total payment (SUM)** and **average payment (AVG)** per customer
- Display the **Top 3 customers** based on total payment

c) Using Python: (2)

- Fetch customer_id and amount
- Perform aggregation to calculate:
 - total payment
 - average payment
- Identify the **Top 3 customers**

d) Measure and compare execution time for: (1)

- SQL query (EXPLAIN ANALYZE)
- Python processing (time module)

Question 6: Using Python (SQLAlchemy class) (5 Marks)

a) Create a **VIEW** named customer_payment_summary that displays: (2)

- customer_id
- total payment (SUM)
- average payment (AVG)

b) Write a query to: (1)

- Retrieve the **Top 5 customers** based on total payment from the view

c) Write a query to: (1)

- Retrieve customers whose **total payment is greater than 150**

d) Use:

- EXPLAIN ANALYZE
- time module

to measure and display execution time of the above queries (1)